Full first-party decompilation of OpenSSL 4.0.1

What this is

A complete, end-to-end decompilation of every first-party function in the OpenSSL 4.0.1 command-line program (libcrypto + libssl + the apps), produced with Ghidra headless. This is the whole-binary counterpart to the targeted blind validation in `../openssl-blind-analysis/`: where that report identified and verified 15 representative functions, this one decompiles the entire first-party surface.

The decompilation itself (a 22.3 MiB C listing) and its compressed artifact are not committed to git, per repository policy on large blobs. What is committed here is the report, a machine-readable manifest with hashes and counts, the full per-function index, and the scripts to reproduce it. The artifact is rebuildable on demand and verifiable against the recorded hashes.

1. Scope and method

- **Scope:** first-party OpenSSL only. The bundled, statically linked glibc and C runtime are excluded. The first-party set is authoritative: it is the defined text symbols (`T`/`t`) from the build's own `libcrypto.a`, `libssl.a`, and the apps objects, giving an allow-list of 15,750 function symbols.
- **Subject:** the symbol-bearing build of `apps/openssl` (unstripped) was decompiled, so the output carries real function names and the first-party filter is exact. This is the same program as the stripped blind subject, just with symbols retained for a useful named listing. Hashes for both are in `manifest.json`.
- **Engine:** Ghidra headless (`analyzeHeadless`), the same decompiler the Vivarium worker wraps. The program was analyzed once and saved to a Ghidra project, then decompiled in bounded, resumable chunks (see section 3 for why bounded chunks were necessary).
- **Output:** one C listing concatenating every decompiled function with a header comment (`name @ address (size)`), plus `index.csv` recording, per function: name, address, size in bytes, decompiled line count, and status (`ok`, `failed`, or `skipped_large`).

Tooling note for reproducibility: this one-off run used the host's Ghidra 10.3.2. The Vivarium worker pins Ghidra 12.1.2, so results via the MCP path could differ slightly. This is recorded in `manifest.json` rather than glossed over.

2. Results

Measure	Value
First-party functions decompiled	15,423
Decompiled cleanly (`ok`)	15,412
Failed (decompiler aborted)	11
Skipped (oversize guard)	0
Success rate	99.93%
Decompiled C	788,037 lines, 22.3 MiB
Compressed artifact	3.3 MiB

The allow-list had 15,750 symbols but 15,423 distinct functions were emitted. The 327 difference is expected: those symbols have no distinct function body in the binary (aliases, weak symbols, or functions the compiler inlined or folded).

Module breakdown (by symbol prefix)

Area	Functions
ossl core (`OSSL_` / `ossl_`)	3,177
SSL/TLS	1,084
EVP	978
ASN.1	855
X509	743
EC	334
RSA	307
BIGNUM	256
PKCS	233
BIO	214
CRYPTO/util	200
apps (subcommands)	48
other	6,994

The 11 functions that did not decompile

`print_out`, `SSL_client_hello_get1_extensions_present`, `SSL_client_hello_get_extension_order`, `custom_exts_copy_conn`, `bn_mod_exp_mont_fixed_top`, `sk_reserve`, `ChaCha20_ssse3`, `ChaCha20_4x`, `DES_cfb_encrypt`, `ossl_ed25519_verify`, `sha256_multi_block`.

These are dominated by heavily-unrolled or SIMD/assembly crypto primitives (`ChaCha20_4x`, `sha256_multi_block`, `bn_mod_exp_mont_fixed_top`, `ossl_ed25519_verify`, `DES_cfb_encrypt`) whose decompilation explodes the decompiler's memory. The memory guards (section 3) made them abort and be recorded as `failed`, rather than crash the whole run. They are a known-hard residue, not a silent gap.

3. Memory-leak investigation (why this took real engineering)

The first attempts to decompile the whole binary repeatedly failed: the host's out-of-band execution daemon was killed by the kernel OOM-killer, four times. The machine is capable of the extraction; the failures were a memory leak in the extraction path, not a hardware limit. Three distinct causes were found, each with direct evidence:

1. **Unbounded native-decompiler growth (the primary leak).** Ghidra runs its decompiler as a separate native process. Reusing a single `DecompInterface` across thousands of `decompileFunction` calls without disposing it grows that native process without bound; it reached roughly 10 GB at about 8,268 functions and triggered the OOM. The fix is to dispose and recreate the decompiler periodically. A controlled 400-function probe with disposal every 100 functions held flat at about 700 MB JVM plus about 400 MB native (around 1.1 GB total), the native process resetting at each disposal boundary instead of climbing.

2. **Pathological single functions.** Even with periodic disposal, one normal-size function (near the exact position the first run died) ballooned the native decompiler to 6.4 GB on its own during a single decompile. A payload cap (`setMaxPayloadMBytes`), a per-function timeout, and an oversize-function guard make such functions abort and be recorded as `failed`, after which memory resets. A resume test confirmed the run crosses the bomb zone with the bomb recorded as a single `failed` entry and memory returning to baseline.

3. **No per-job memory isolation in the execution framework.** These runs executed under the host's out-of-band execution daemon (cotd), where every job shares the daemon's cgroup with no per-job memory bound (no `setrlimit`, no transient scope, `MemoryMax=infinity`). A runaway job's memory therefore counts against the shared daemon cgroup, so the kernel OOM-killer takes down the whole daemon (and every concurrent job with it) rather than just the offending job. This is a reliability gap distinct from the Vivarium worker's own bounded lifecycle (kill-on-evict with a verified wipe, ADR-002). Filed as claude-tools#11; the fix is a per-job memory cap (`setrlimit` via `preexec_fn`, a transient `systemd-run` scope, or a daemon-level `MemoryMax`/`OOMScoreAdjust`). Note: on a cot-initiated timeout the runner already kills the job's process group, so this is specifically about an unbounded job triggering a daemon-wide kernel OOM, not orphaned processes.

Incidental finding: `analyzeHeadless` hardcodes `MAXMEM=2G` as a plain assignment (not `${MAXMEM:-2G}`), so an inherited `MAXMEM` environment variable is silently ignored and the JVM heap is fixed at 2 GB. This is not the leak (it keeps the heap small); the runaway memory was native, off-heap, in the decompiler process.

With causes 1 and 2 fixed in the harness and cause 3 worked around by keeping the job's own memory well under the host ceiling, the full remaining decompilation (about 7,200 functions) completed in 124 seconds with bounded memory and no OOM, versus tens of minutes of OOM-looping before. The guards applied are recorded in `manifest.json`.

4. Files in this directory

- `REPORT.md`, `REPORT.pdf` - this report.
- `manifest.json` - machine-readable subject hashes, tooling, results, the failed list, module breakdown, and the memory guards used.
- `index.csv` - the full per-function index (15,423 rows: name, address, size, decompiled lines, status). Queryable without the full listing.
- `scripts/ExportChunk.py` - the Ghidra Jython export script with the memory guards.
- `scripts/first-party-allowlist.txt` - the 15,750 first-party symbol names.
- `reproduce.md` - exact commands to rebuild the decompilation.

Not committed (rebuildable; verify against `manifest.json` hashes): `openssl_firstparty_decompiled.c` (22.3 MiB) and `openssl-firstparty-decompiled.tar.gz` (3.3 MiB).

5. Reproducing

See `reproduce.md`. In outline: build OpenSSL 4.0.1 (the build script in `../openssl-blind-analysis/build-openssl-fixture.sh` produces the matching binary; keep the unstripped `apps/openssl`), build the first-party allow-list from the archives, analyze and save the program once with `analyzeHeadless`, then run `scripts/ExportChunk.py` in bounded chunks (disposal every 50, payload cap 30 MB, timeout 30 s) against the saved project until no first-party functions remain.